A Progress (Review-1) REPORT ON

AI-POWERED SMART WATER MANAGEMENT AND RATIONING SYSTEM

(Academic Year: 2026-27)

SUBMITTED BY

PRN	Student Name
23070123055	Gayatri Apotikar
23070123058	Hardik Dhalla
23070123102	Prashant Bajpai
23070123145	Trijal Burman

Under the Guidance of

Dr. D. Marshiana
Assistant Professor, Department of Electronics and Telecommunication Engineering

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

[(A CONSTITUENT OF SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)]

Pune – 412115

Date of Submission: August 2026

I. INTRODUCTION

1.1 Background

Water scarcity is one of the most pressing global challenges of the 21st century. According to the United Nations, over 2 billion people currently live in countries experiencing high water stress, and this number is projected to rise significantly by 2050 due to population growth, industrialisation, and climate change. Municipal water distribution networks in developing nations suffer from high non-revenue water losses — often exceeding 30–40% — due to leaks, theft, and poor monitoring. Existing infrastructure relies heavily on manual meter reading, periodic inspections, and reactive maintenance, all of which are slow, expensive, and error-prone.

The emergence of Internet of Things (IoT) technology, low-cost microcontrollers, and open-source time-series databases presents an opportunity to build affordable, real-time water management systems that can autonomously monitor, predict, and respond to supply conditions without human intervention. This project leverages these technologies to build an end-to-end smart water management and rationing system that is scalable, containerised, and AI-ready.

1.2 Problem Statement

Current water distribution systems in residential and small industrial settings lack continuous real-time monitoring of tank levels and consumption rates. Administrators have no visibility into how quickly water is being consumed, when depletion is likely to occur, or whether abnormal flow patterns indicate leakage. Manual rationing decisions are subjective and reactive, often leading to either wasteful over-supply or inequitable under-supply. There is a clear need for an automated system that can sense supply conditions, predict depletion, detect anomalies, and enforce rationing decisions without manual intervention.

1.3 Objectives

1. Design and build a physical IoT prototype that measures water tank level and flow rate in real time using an HC-SR04 ultrasonic sensor and a YF-S201 flow sensor connected to an ESP32 microcontroller.

2. Implement an MQTT-based telemetry pipeline that transmits sensor readings to a FastAPI backend over a Mosquitto broker.

3. Store all telemetry and control decisions in an InfluxDB 2.7 time-series database optimised for Grafana Flux queries.

4. Develop an explainable AI control algorithm that estimates time-to-depletion and

commands a solenoid valve through five operating modes: NORMAL, WARNING, RATIONING, CRITICAL, and RECOVERY.

5. Provision a Grafana dashboard with eight real-time panels displaying tank level, flow rate, depletion prediction, and system mode.

6. Containerise the complete stack using Docker Compose for one-command deployment.

1.4 Scope of the Project

The project covers the complete stack from physical sensor hardware through to a live monitoring dashboard. The scope includes the ESP32 firmware for sensor reading and MQTT publishing, the FastAPI backend implementing the state-machine control loop and InfluxDB writer, Grafana dashboard with auto-provisioned datasource and pre-built panels, Docker Compose orchestration of all four services (Mosquitto, InfluxDB, FastAPI, Grafana), a simulation tool for desktop testing without physical hardware, and startup documentation for easy deployment. The scope excludes cloud deployment on AWS/Azure and multi-tank network management, which are identified as future work.

II. PROJECT PROGRESS DETAILS

2.1 Literature Review

A systematic survey of existing IoT-based water management systems was conducted. Key findings include: (a) Rajeswari et al. (2019) demonstrated a Raspberry Pi ultrasonic sensor system with SMS alerts but no time-series database. (b) Kumar & Singh (2020) implemented timer-based Arduino rationing with solenoid valves but lacked flow sensing and depletion prediction. (c) Patel et al. (2021) showed long-range MQTT delivery over LoRaWAN but lacked a real-time dashboard. (d) Gonzalez et al. (2023) confirmed InfluxDB and Grafana suitability for high-throughput industrial IoT monitoring. (e) Zhao et al. (2024) demonstrated edge AI anomaly detection with 96% leak detection recall. The survey identifies five gaps addressed in this project: lack of containerised deployment, no explainable AI control loop, no integrated time-series backend, no provisioned dashboards, and absence of a MQTT-to-REST bridge.

2.2 System Architecture

The system follows a four-tier architecture: (1) Sensing and Actuation — HC-SR04 ultrasonic sensor and YF-S201 flow sensor connected to an ESP32-WROOM-32 microcontroller, with a 12 V DC normally-closed solenoid valve for flow control. (2) Edge Processing and Communication — The ESP32 samples sensors at 1 Hz, computes tank level percentage and flow rate, and publishes JSON telemetry to the watertank/telemetry

MQTT topic. (3) Backend Analytics and Control — A FastAPI 0.115 application subscribes to MQTT, applies the state-machine control algorithm, writes results to InfluxDB 2.7, and publishes valve commands to watertank/command. (4) Visualisation — Grafana 11.4 queries InfluxDB directly using Flux queries and displays eight dashboard panels refreshed every five seconds.

2.3 Hardware Components

The physical prototype comprises: ESP32-WROOM-32 (240 MHz dual-core, Wi-Fi + BT) as the central microcontroller; HC-SR04 ultrasonic sensor (2–400 cm, ±3 mm accuracy) for tank level measurement via echo time-of-flight; YF-S201 Hall-Effect flow sensor (1–30 L/min, ±10%) for volumetric flow measurement via pulse counting; a 12 V DC normally-closed solenoid valve for supply control, PWM-driven via GPIO; a 5 V submersible mini pump to circulate water; and transparent acrylic containers with vinyl tubing to make water movement visible during demonstration.

2.4 Software Stack and Docker Infrastructure

All backend services are containerised in a single docker-compose.yml file under the infra/ directory. Four services are orchestrated: (1) Eclipse Mosquitto 2.x — MQTT broker running on port 1883 with anonymous access and no persistence. (2) InfluxDB 2.7 — time-series database auto-initialised on first boot via DOCKER_INFLUXDB_INIT_* environment variables, creating the water-system organisation, telemetry bucket, and admin token. (3) FastAPI 0.115 + Uvicorn + Python 3.12 — backend API built from a Dockerfile, exposing five REST endpoints and subscribing to MQTT. (4) Grafana 11.4.0 — dashboard service reading provisioning YAML files at startup to auto-register the InfluxDB datasource and load the dashboard JSON, requiring zero manual UI configuration.

2.5 AI Control Algorithm

The FastAPI backend implements an explainable state-machine control loop. On every telemetry reading, the engine computes a rolling five-sample average of past flow rates to estimate predicted demand, then calculates Time-to-Depletion (ETA) = Usable Volume (L) ÷ Predicted Flow (L/min), where Usable Volume = Current Volume − 10% minimum safe reserve. The system transitions through five modes: NORMAL (ETA ≥ 120 min, valve 100%), WARNING (60 ≤ ETA < 120 min, valve 100%, alert issued), RATIONING (ETA < 60 min, valve throttled to 20–70% based on remaining volume), CRITICAL (level < 10%, valve closed 0%), and RECOVERY (automatic restoration when ETA recovers above 120 min). Each decision is logged with a human-readable reason string and priority

flag for full auditability.

2.6 Grafana Dashboard

The Grafana dashboard is fully auto-provisioned via two YAML files in grafana/provisioning/. The datasource YAML registers InfluxDB in Flux mode with a pinned UID and the admin token. The dashboard JSON defines eight panels: (1) Tank Level % gauge (0–100%, colour thresholds at 10%/25%/50%); (2) Flow Rate gauge (0–20 L/min); (3) Time to Depletion stat panel with colour-coded background; (4) System Mode stat panel; (5) Tank level over time timeseries; (6) Flow rate and valve position dual-series timeseries; (7) Volume in litres timeseries; (8) Predicted depletion time trend. All panels query the telemetry bucket via Flux and auto-refresh every 5 seconds.

2.7 Implementation and Testing

End-to-end testing was performed using the Python simulator (tools/simulate.py), which posts a sequence of five telemetry readings spanning all five operating modes. All state machine transitions were verified across three consecutive cycles (15 readings, 138 InfluxDB data points per field). InfluxDB storage was confirmed using a Flux count query returning 138 records per measurement field. Grafana displayed all eight panels with live data. The following challenges were resolved during implementation: (a) InfluxDB login failure due to old bcrypt password in persistent volume — fixed using the influx user password CLI with the known admin token; (b) Grafana "No data" on all panels — caused by unresolved ${DS_INFLUXDB} template variable; fixed by hardcoding datasource UID P951FEA4DE68E13C5 in all panels; (c) Docker permission denied — fixed via sudo chmod 666 /var/run/docker.sock; (d) MQTT bridge silently failing during broker startup — resolved with OSError catch so the API remains operational; (e) InfluxDB writer blocking the control loop — resolved with 3 s timeout and non-blocking exception handling.

2.8 Work Completed So Far

✓ Complete FastAPI backend with state-machine control loop, InfluxDB writer, and MQTT bridge

✓ Docker Compose stack with four services (Mosquitto, InfluxDB, FastAPI, Grafana) — one-command deployment

✓ Grafana auto-provisioned datasource and eight-panel dashboard verified with live data

✓ Python simulator for desktop testing across all five operating modes

✓ Firmware architecture designed for ESP32 (Arduino C++) — MQTT publish and solenoid control

✓ Startup documentation (start.sh, START.md, Makefile) for reproducible deployment

✗ Live ESP32 hardware integration and solenoid valve closed-loop test — pending

✗ ARIMA/Prophet demand forecasting model — planned for Review-2

✗ Leak detection using dual-flow-sensor volume balance — planned for Review-2

2.9 Work Planned (Review-2 Onwards)

1. Complete ESP32 firmware, solder connections, and perform live hardware test with solenoid valve.

2. Integrate dual YF-S201 flow sensors for volume-balance leak detection.

3. Train ARIMA or Facebook Prophet model on InfluxDB historical data for 24-hour demand forecasting.

4. Extend to multi-tank network management with per-device_id supply allocation.

5. Deploy on AWS IoT Core (MQTT) with cloud-hosted InfluxDB and Grafana for 24/7 availability.

6. Develop a mobile dashboard (React Native / Flutter) with push notifications for CRITICAL events.

III. REFERENCES

Rajeswari S. et al., "IoT Based Smart Water Monitoring System," International Journal of Engineering and Technology, vol. 11, no. 3, 2019.

Kumar A. and Singh R., "Automated Water Rationing System Using Arduino and GSM," IJARCCE, vol. 9, no. 6, 2020.

Patel R. et al., "Smart Water Management with LoRaWAN and MQTT," IEEE Internet of Things Journal, vol. 8, no. 14, 2021.

Chen X. et al., "LSTM-Based Urban Water Demand Forecasting," Water Resources Research, vol. 58, 2022.

Sharma P. et al., "Real-Time Water Level Monitoring Using ESP8266 and Cloud IoT," IOSR-JECE, vol. 18, no. 2, 2023.

Gonzalez M. et al., "InfluxDB and Grafana for Industrial IoT Monitoring," IEEE IECON, 2023.

Zhao Y. et al., "Edge AI for Anomaly Detection in Smart Water Networks," ACM SenSys, 2024.

Eclipse Foundation, "Eclipse Mosquitto MQTT Broker," Version 2.0. [Online]. Available: https://mosquitto.org/

InfluxData Inc., "InfluxDB 2.7 Documentation." [Online]. Available:

https://docs.influxdata.com/influxdb/v2.7/

Grafana Labs, "Grafana 11.4 Documentation." [Online]. Available: https://grafana.com/docs/

Tiangolo, "FastAPI Framework Documentation." [Online]. Available: https://fastapi.tiangolo.com/

Espressif Systems, "ESP32 Technical Reference Manual," Version 5.0, 2023.

ITU/UNESCO, "The United Nations World Water Development Report 2023: Partnerships and Cooperation for Water," UNESCO, Paris, 2023.

Docker Inc., "Docker Compose Reference Documentation." [Online]. Available: https://docs.docker.com/compose/

Eclipse Paho, "Eclipse Paho MQTT Python Client," Version 2.1. [Online]. Available: https://www.eclipse.org/paho/